# Blatt 09: Zoo (Generics)

Carsten Gips (HSBI)

**Unterlagen**

- PDF-Version

---

Sie finden in den Vorgaben im Package `zoo` einige Interfaces und Klassen, mit denen man einen Zoo modellieren kann. Klonen Sie das Vorgabe-Repo und laden Sie das Projekt als Gradle-Projekt in Ihre IDE.

## 1 Nicht-generische Klassen

Sie finden einige Interfaces, die das Interface `Animal` erweitern, beispielsweise `Fish`, `Reptile`, `Cat`, ...

Erstellen Sie für jedes dieser Interfaces jeweils zwei Java-Klassen, die konkrete Mitglieder der jeweiligen Tier-Klasse bzw. -Familie[1] repräsentieren und die jeweils das Interface implementieren. Beispielsweise könnte eine Forelle eine Art in der Klasse der Fische sein, d.h. die Java-Klasse `Trout` könnte das Interface `Fish` implementieren.

Diese Java-Klassen sollen nicht-generisch sein. Die `move()`-Methode soll einfach einen String auf der Konsole ausgeben, wie sich dieses Tier bewegen würde. Mit der `getName()`-Methode soll ein kurzer Name, beispielsweise der Tierklasse, zurückgegeben werden.

*Hinweis*: Die mit **`default`** markierten Methoden in `Fish`, `Mammal` und `Reptile` brauchen Sie in Ihren Klassen nicht implementieren. Wir werden später im Semester noch über die sogenannten "Default-Methoden" sprechen.

## 2 Generische Klassen

Um Gehege zu modellieren, erstellen Sie in eine generische Klasse `Habitat` mit einer Typ-Variablen. Stellen Sie durch geeignete Beschränkung der Typ-Variablen sicher, dass nur Gehege mit von `Animal` abgeleiteten Typen gebildet werden können.

Es sollen (mindestens) folgende Methoden existieren:

- `void add(Animal animal)` zur Aufnahme eines Tieres in das Gehege
- `void remove(Animal animal)` um ein Tier aus einem Gehege zu entfernen
- `String getName()` um den Namen des Geheges zu bestimmen

*Hinweis*: Der Parametertyp `Animal` in den Methoden muss entsprechend angepasst werden.

Jedes einzelne Tier im Gehege kann maximal einmal vorkommen. Begründen Sie die Wahl der Datenstruktur.

Legen Sie in Ihrer `main()`-Methode mindestens zwei konkrete Gehege als Instanzen der neuen generischen Klasse `Habitat` an. Dabei sollen in diesen konkreten Gehegen nur jeweils verschiedene Tiere einer "Art" (beispielweise Löwen, Hamster, ...) vorkommen. Fügen Sie einige passende Tiere in die beiden Gehege ein.

[1] Ok, wir machen hier Informatik. Vermutlich ist die Biologie nicht ganz korrekt ;-)

## 3 Generische Klassen reloaded

Für die Repräsentation eines Zoologischen Gartens mit mehreren verschiedenen Gehegen erstellen Sie nun eine generische Klasse `Zoo` mit einer Typ-Variablen. Stellen Sie durch geeignete Beschränkung der Typ-Variablen sicher, dass nur Zoos mit von `Habitat` abgeleiteten Typen gebildet werden können.

Ein Zoo soll (mindestens) folgende Methoden besitzen:

- `void build(Habitat habitat)` zur Aufnahme eines neuen Geheges in den Zoo
- `void abandon(Habitat habitat)` um ein Gehege aus dem Zoo zu entfernen
- `void visitAllHabitats()` um die Gehege in der Reihenfolge des Hinzufügens zu besuchen (d.h. ihren Namen auf der Konsole auszugeben)

*Hinweis*: Der Parametertyp `Habitat` in den Methoden muss entsprechend angepasst werden.

Jedes Gehege gibt es pro Zoo nur maximal einmal, und die Reihenfolge des Errichtens definiert eine Ordnung auf den Gehegen. Begründen Sie die Wahl der Datenstruktur.

Legen Sie in Ihrer `main()`-Methode einen konkreten Zoo als Instanz der neuen generischen Klasse `Zoo` an. Dieser Zoo soll einige Gehege für verschiedene Tiere beinhalten.

## 4 Ableiten nicht-generischer Klassen

Leiten Sie von `Zoo` eine nicht-generische Klasse `Aquarium` ab. Aquarien können nur mit Gehegen angelegt werden, deren Tiere vom Typ `Fish` (oder abgeleitet) sind.

Legen Sie in Ihrer `main()`-Methode ein konkretes Aquarium als Instanz der neuen nicht-generischen Klasse `Aquarium` an und gruppieren Sie darin verschiedene Fisch-“Gehege”.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** c1867fd (tooling: remove numbering of homework tasks, 2025-04-23)